UIT University

BACHELOR OF SCIENCE (COMPUTER SCIENCE)

Spring 2026

CSC-205 - Theory of Automata

PROJECT REPORT

➢ **TITLE:**

Interactive DFA-Based Email & Password Validation System

➢ **GROUP MEMBERS:**

- Muhammad Fahaz Khan (24SP-047-CS)
- Hassan-Ul-Arefeen (24SP-010-CS)
- Muhammad Fawwad Siddiqui (24SP-035-CS)
- Saim-Ul-Haq (24SP-006-CS)

1. Project Title and Objective

1.1 Project Title

Interactive DFA-Based Email & Password Validation System

1.2 Project Objective

The primary objective of this project is to bridge the gap between abstract computational mathematics and practical software engineering. While modern applications heavily rely on Regular Expressions (Regex) for user input validation, the underlying execution of these rules is often hidden from developers and users. This project aims to build a responsive desktop application that not only validates strict string structures (such as email formats and complex password policies) but visually demystifies the process by mapping the strings dynamically through a Deterministic Finite Automaton (DFA). By creating an interactive "Automata Laboratory," the software provides step-by-step visualizations, live Turing-style input tape tracking, and formal transition tables to prove language acceptance in real-time.

2. Problem Statement

In the context of Automata Theory, the problem this project solves is the "Black Box" nature of standard string validation algorithms.

When a developer uses a standard Regex library to validate an email or a password, the computer translates that expression into a finite state machine, executes it, and returns a binary True/False (Accepted/Rejected) response. However, for students and researchers of computational theory, a binary output is insufficient. The theoretical problem is demonstrating *how* and *why* a specific string is accepted or rejected based on formal language constraints.

If a password fails validation, the theoretical question is: *At which specific state in the machine did the input halt, and what symbol caused the machine to transition into a Dead State?*

This project solves this by constructing an interactive simulation environment that exposes the deterministic state machine. It solves the problem of theoretical visualization by rendering the mathematical model dynamically, proving language acceptance mathematically character-by-character rather than simply relying on background software libraries.

3. Applied Concepts

This project applies several core concepts from Automata Theory, strictly relying on the principles of **Deterministic Finite Automata (DFA)** and **Regular Languages**.

3.1 Regular Expressions (Regex) and Regular Languages

The validation logic strictly defines two formal languages:

1. $\mathbf{L}_{email}$ The language of all valid email addresses formatted as username@domain.extension.
2. $\mathbf{L}_{password}$ The language of all strings $\boldsymbol{\omega}$ where $|\boldsymbol{\omega}| \geq \mathbf{8}$, and $\boldsymbol{\omega}$ contains at least one symbol from the alphabets of uppercase letters, lowercase letters, and numeric digits.

Because both languages can be expressed via Regular Expressions, they are fundamentally Regular Languages. According to Kleene's Theorem, if a language is defined by a Regular Expression, there exists a Finite Automaton that recognizes it.

3.2 NFA vs. DFA Transition Methodology

A Regular Expression is typically converted first into a Non-Deterministic Finite Automaton (NFA) using Thompson's Construction. However, NFAs allow for **$\boldsymbol{\epsilon}$-transitions** (empty string jumps) and multiple active states simultaneously, which is highly inefficient for algorithmic simulation **($\boldsymbol{O(2^n)}$ worst-case time complexity).**

To ensure optimal, real-time performance $\boldsymbol{O(n)}$, this project applies the Subset Construction Algorithm (Powerset Construction) theory to convert the implicit NFA logic into a strict **DFA**. In our implemented DFA, every state has exactly one defined transition for every possible input symbol, meaning the machine is in exactly one state at any given time.

3.3 The 5-Tuple Mathematical Model

Our implementation strictly models the formal mathematical definition of a DFA, represented by the 5-tuple $\boldsymbol{M = (Q, \Sigma, \delta, q_0, F)}$:

- $\boldsymbol{Q}$: The finite set of states **(e.g., q_0, q_1, q_{dead})**.
- $\boldsymbol{\Sigma}$: The input alphabet (ASCII characters).
- $\boldsymbol{\delta}$: The transition function $(\boldsymbol{Q \times \Sigma \rightarrow Q})$, mapped internally via Python dictionaries.
- $\boldsymbol{q_0}$: The start state.
- $\boldsymbol{F}$: The set of final/accepting states.

4. Methodology and Logic

The project's methodology centers on mapping deterministic transition matrices into executable Python code and corresponding visual nodes.

4.1 Automata Construction Logic

Instead of utilizing standard expression parsing, we mathematically predetermined the required states for our languages. The logic flows systematically:

1. **Initialization:** The machine initializes at state q_0 and reads the first symbol from the input string (Input Tape).
2. **Execution Loop:** The simulation traverses the string. For each character, the transition function $\boldsymbol{\delta(q_{current}, symbol)}$ is evaluated to return the next state.
3. **Halt & Verification:** Upon reading the end of the string, if $\boldsymbol{q_{current} \in F}$ in $\boldsymbol{F}$, the string is **Accepted**. If $\boldsymbol{q_{current} \notin F}$ (or if the machine previously fell into a trap state), the string is **Rejected**.

4.2 State Transition Tables

Below are the mathematical state transition matrices manually designed and implemented within the software's engine.

Table 1: Email Verification DFA Transition Matrix

Current State (Q)	Input: letter/num	Input: @	Input: .
$\rightarrow q_0$ (Start)	q_1	Dead	Dead
q_1	q_1	q_2	Dead
q_2	q_3	Dead	Dead
q_3	q_3	Dead	q_4
q_4	q_5 (Accept)	Dead	Dead
q_5 (Accept)	q_5	Dead	Dead
Dead State	Dead	Dead	Dead

Table 2: Multi-Condition Password Verification Matrix

(Note: U = Uppercase, L = Lowercase, D = Digit. Length requirement of ≥ 8 *is tracked dynamically alongside these state changes)*

Current State (Q)	Input: U	Input: L	Input: D
$\rightarrow q_0$ (Start)	q_U	q_L	q_D
q_U	q_U	q_{UL}	q_{UD}
q_L	q_{UL}	q_L	q_{LD}
q_D	q_{UD}	q_{LD}	q_D
q_{UL}	q_{UL}	q_{UL}	q_{ULD}

Current State (Q)	Input: U	Input: L	Input: D
q_{UD}	q_{UD}	q_{ULD}	q_{UD}
q_{LD}	q_{ULD}	q_{LD}	q_{LD}
q_{ULD} (Accept)	q_{ULD}	q_{ULD}	q_{ULD}

5. Code Explanation

5.1 Alignment with Automata Theory

The Python source code serves as a direct virtualization of a DFA machine. Rather than using unstructured logic, the application uses dictionaries to act as the transition function δ, tracking the step-by-step history of the input pointer moving across the string.

5.2 Description of Key Classes and Logic Flow

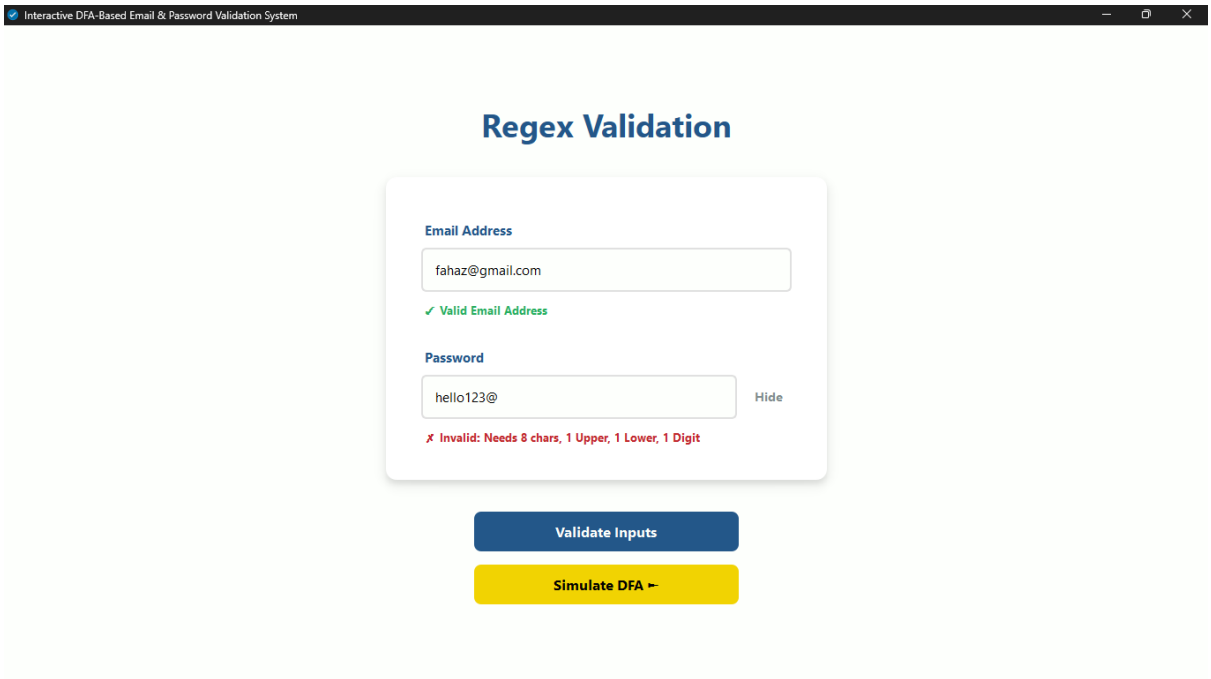
- **simulate_email_dfa(string) and simulate_password_dfa(string)**: These core backend functions process the transition tables. They utilize a for-loop to mimic the Read Head of a Finite Automaton moving one cell at a time across the input string.
- **SimulatorPage Class (PyQt6)**: This handles the entire laboratory UI. It manages the graphical canvas, updates the Input Tape label, and contains the logic for manual machine playback (Pause, Step Forward, Step Back).
- **graph_generator.py**: This script translates theoretical node maps into mathematical vectors using the Graphviz library, rendering high-resolution 300 DPI frames of every possible state highlight.

5.3 Implementation of Theoretical Concepts

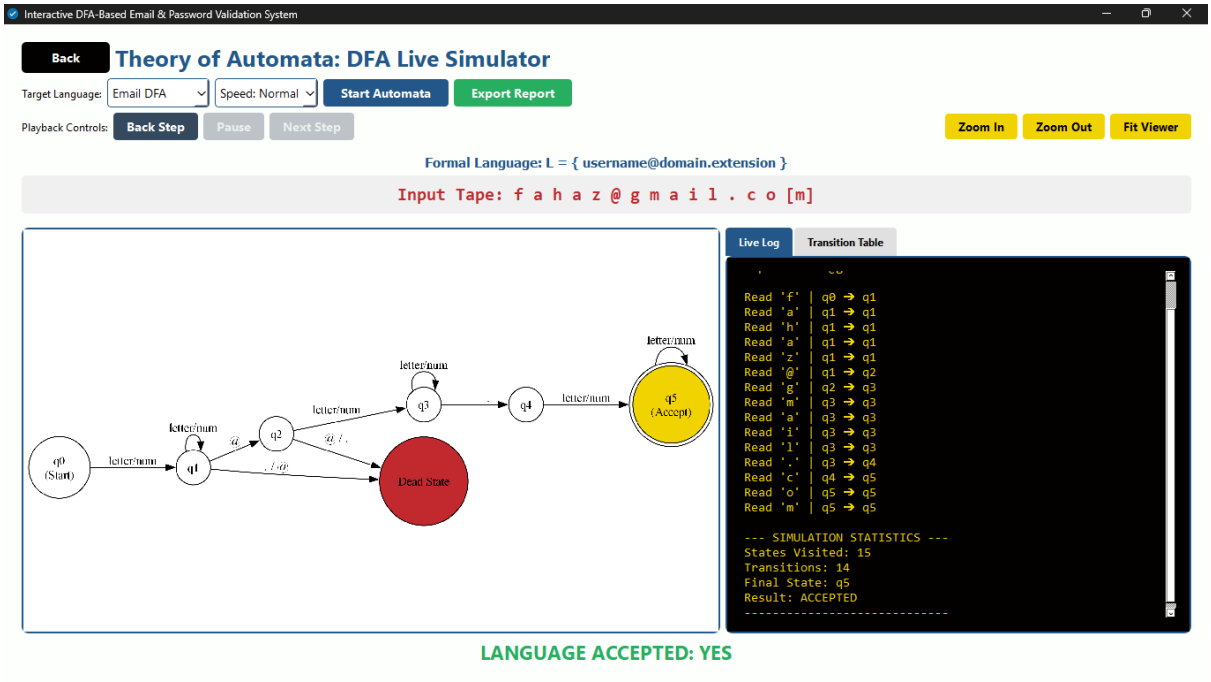
- **The Input Tape (update_tape method):** In theory, a machine reads from an infinite tape. In our code, this is implemented via an index pointer on a string array. The UI dynamically highlights the current index using brackets (e.g., f a h a [z] @ g m a i l . c o m) to simulate the Read Head.
- **Transition Execution (animate_next_step method):** This function pulls the next character and looks up its destination state in the dictionary map. It prints the mathematical step to the terminal log in the formal syntax: Read 'z' | q1 ➔ q1.
- **State Highlighting ("Flipbook" Logic):** To simulate the active state without blocking the application thread, the software instantly swaps pre-rendered Graphviz .png files, visually confirming that the machine is strictly in one single state at a time (proving Determinism).

6. Screenshots of Simulation and Output

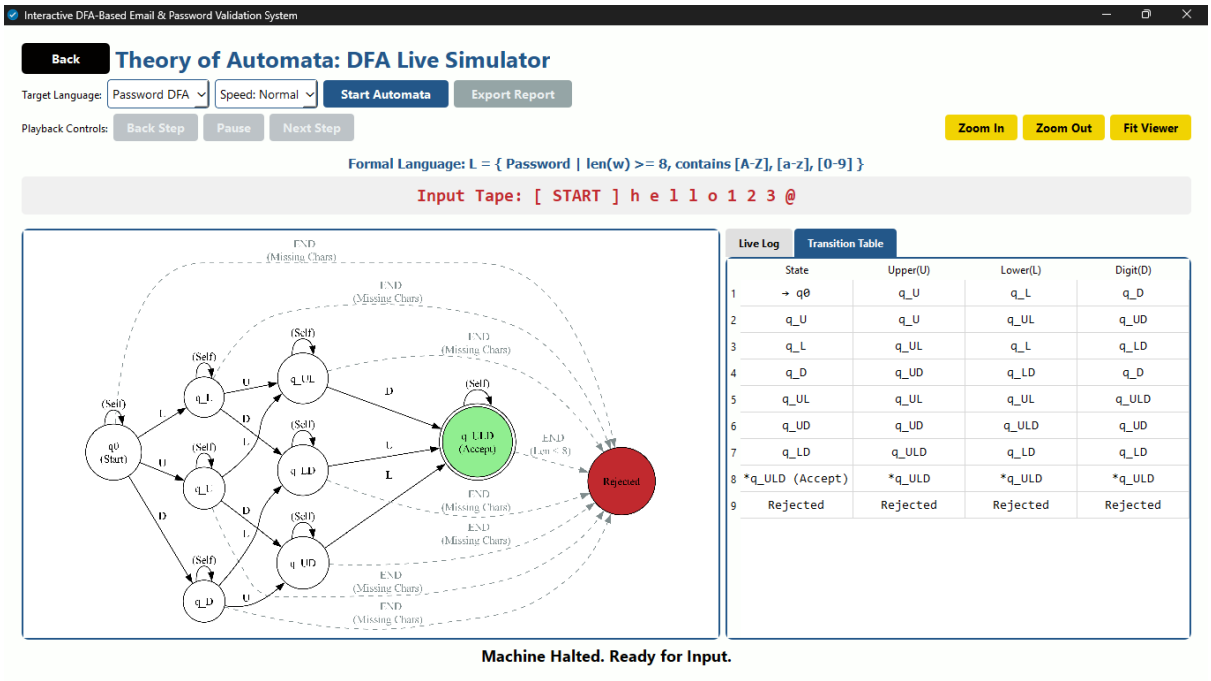
- **Figure 6.1: Regex Validation Form**



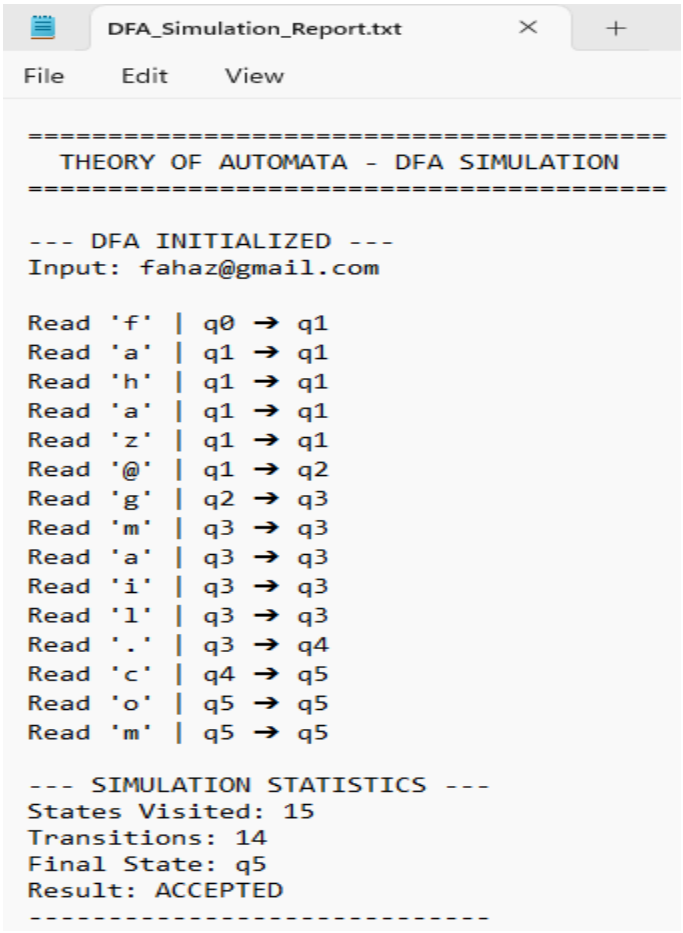
- **Figure 6.2: DFA Simulator Laboratory UI**



- **Figure 6.3: Formal Transition Table View**



- **Figure 6.4: Exported Simulation Log**



7. Challenges and Resolutions

Developing a graphical automata simulator presented several unique software engineering challenges:

7.1 Graphical Rendering Bottlenecks

- **Challenge**: Initially, attempting to mathematically generate new Graphviz nodes dynamically during the animation loop caused severe UI lag. Because PyQt6 runs on a single thread, generating images blocked the UI thread, causing the program to freeze between state transitions.
- **Resolution**: Implemented a pre-rendering "Flipbook" architecture. A separate script computes and saves every possible state-highlighted image before the application launches. During runtime, the machine merely loads these images instantly from memory, achieving zero-lag deterministic transitions.

7.2 Handling Invalid and Null Inputs

- **Challenge**: If a user clicked "Simulate DFA" on an empty string, the internal Read Head loop would throw an index error, as $\boldsymbol{\Sigma}$ cannot process a null tape in a strict DFA without an explicit $\boldsymbol{\varepsilon}$**-transition**.
- **Resolution**: Implemented strong pre-validation checks (**if not input:**). The system now halts execution and prevents the simulation buttons from spawning unless a finite, non-zero string is supplied to the Input Tape.

7.3 Responsive UI and Layout Squeezing

- **Challenge**: The inclusion of playback controls, speed dropdowns, and export buttons caused layout overlapping on smaller monitors, resulting in cut-off text on standard 1080p displays.
- **Resolution**: Refactored the UI using horizontal layout splitters (**QHBoxLayout**). The controls were divided logically into two specific rows Configuration Tools on row one, and Machine Playback on row two guaranteeing 100% responsiveness without losing data visibility.

8. Conclusion and Observations

This project successfully proves that abstract computational mathematics can be translated into functional, visually engaging software tools. By building this Interactive DFA-Based Validation System, we observed several key realities regarding Automata Theory in software engineering:

1. **Theoretical Precision:** We observed that strict DFA transition maps are inherently safer and less prone to memory leaks than unbounded regular expressions because every single state execution path, including "Dead States", is rigorously defined and mapped.
2. **Educational Utility:** Creating an interactive Input Tape and displaying step-by-step Terminal Logs significantly improved the debugging process. It allowed us to instantly identify exactly which character caused a string to fail, an insight entirely missing from standard boolean Regex libraries.
3. **Algorithmic Efficiency:** By finalizing the logic strictly as a DFA (avoiding NFA backtracking), the application ensures linear time complexity $\boldsymbol{O(n)}$ proportional only to the length of the string, ensuring maximum performance regardless of how complex the password policy becomes.